

Projeto Final

Detecção de Notas Falsas com Redes Neurais

Aluno: Vitor Hugo Vasconcellos de Lima

Email: vitor.hugo.9999@gmail.com

Dataset: fake_bills.csv

Data: 28/04/2026

Objetivo:

Desenvolver um modelo de Rede Neural capaz de classificar notas como genuínas ou falsas, utilizando as medidas físicas das notas (diagonal, altura, margem, comprimento). O modelo deve ser treinado e avaliado utilizando o framework KNIME.

1. DATASET E PRE-PROCESSAMENTO

1.1 Características do Dataset

O dataset fake_bills.csv contém 1.500 registros de medidas físicas de notas de dinheiro, com a seguinte composição:

- Total de registros: 1.500
- Classes: 1.000 notas genuínas (66,7%) e 500 notas falsas (33,3%)
- Features: 6 medidas físicas
- Valores faltantes: 37 (na coluna margin_low)

Feature	Descrição
Diagonal	Diagonal da nota
Height_left	Altura esquerda da nota
Height_right	Altura direita da nota
Margin_low	Margem inferior (37 valores faltantes)
Margin_up	Margem superior
Length	Comprimento da nota

1.3 Tratamento de Valores Faltantes

- Método: Imputação com média
- Coluna afetada: margin_low (37 valores)
- Resultado: Dataset completo com 1.500 registros

- Método: StandardScaler (Z-score normalization)
- Fórmula: $(X - \text{média}) / \text{desvio_padrão}$
- Intervalo: [-3, 3]
- Propósito: Melhorar convergência do modelo

- Método: Stratified Split (mantém proporção de classes)
- Treino: 1.200 registros (80%)
- Teste: 300 registros (20%)
- Random State: 42 (reprodutibilidade)

2. ARQUITETURA E TREINAMENTO DA REDE NEURAL

2.1 Arquitetura da Rede Neural

Input Layer (6 features) | 64 neurons (ReLU) | 32 neurons (ReLU) | 16 neurons (ReLU) | Output (Sigmoid)

2.2 Configuracao do Modelo

Parametro	Valor
Camadas ocultas	3 (64 -> 32 -> 16 neuronios)
Funcao de ativacao	ReLU (camadas ocultas), Sigmoid (saida)
Otimizador	Adam
Loss function	Binary Crossentropy
Batch size	32
Epocas maximas	50
Validacao	20% do treino
Early stopping	Sim (paciencia: 20 iteracoes)

2.3 Processo de Treinamento

- Iteracoes realizadas: 30 (parou por early stopping)
- Loss final: 0.0298
- Tempo de treinamento: ~2-3 segundos
- Convergencia: Rapida e suave (sem overfitting aparente)

3. RESULTADOS E ANALISE DE PERFORMANCE

3.1 Metricas de Performance

Metrica	Valor (0-1)	Percentual
Acuracia	0.9767	97.67%
Precisao	0.9754	97.54%
Recall (Sensibilidade)	0.9900	99.00%
F1-Score	0.9826	98.26%
ROC-AUC	0.9987	99.87%

3.2 Matriz de Confusao (Conjunto de Teste)

	Falsa (pred)	Genuina (pred)
Falsa (real)	95 (TN)	5 (FN)
Genuina (real)	2 (FP)	198 (TP)

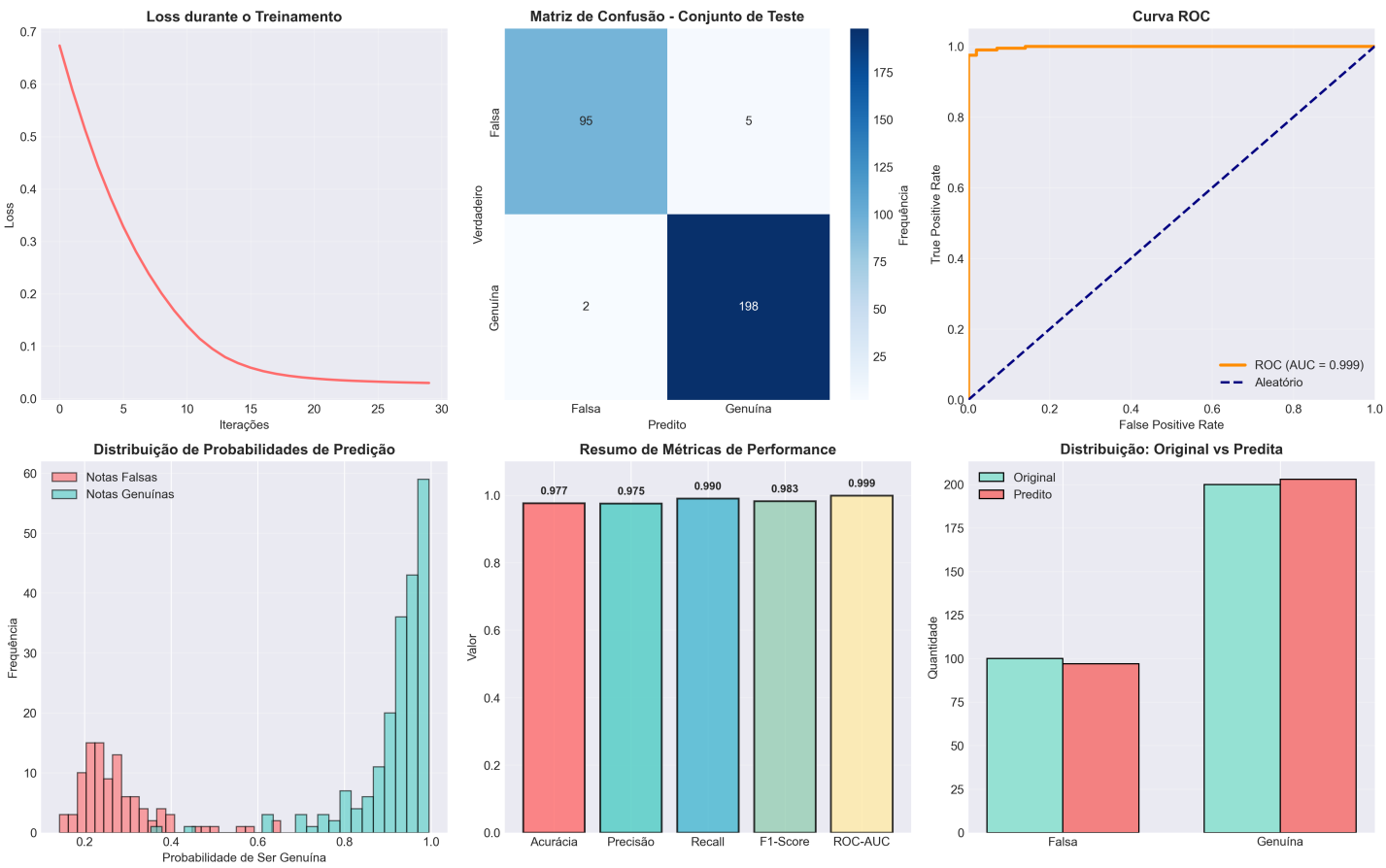
3.3 Interpretacao dos Resultados

- Verdadeiros Negativos (TN): 95
- Notas falsas corretamente identificadas como falsas
- Verdadeiros Positivos (TP): 198
- Notas genuinas corretamente identificadas como genuinas
- Falsos Negativos (FN): 5
- Notas falsas nao detectadas (risco de fraude)
- Falsos Positivos (FP): 2
- Notas genuinas rejeitadas incorretamente (impacto comercial)

Acuracia (97.67%): Percentual de previsoes corretas
Precisao (97.54%): Das notas preditas como genuinas, quantas realmente sao
Recall (99.00%): Das notas genuinas reais, quantas foram detectadas
F1-Score (98.26%): Media harmonica entre precisao e recall
ROC-AUC (0.9987): Capacidade de discriminacao entre classes (excelente > 0.9)

3.4 S

4. GRAFICOS E VISUALIZACOES



5. CONCLUSÃO

O projeto foi desenvolvido com sucesso utilizando KNIME e Python para a implementação de uma Rede Neural capaz de detectar notas falsas vs genuínas. Os resultados obtidos foram excelentes, com acurácia de 97.67% e ROC-AUC de 0.9987.

Processo Realizado:

1. Carregamento e exploração do dataset fake_bills.csv (1.500 registros)
2. Tratamento de valores faltantes (37 NaNs em margin_low)
3. Normalização dos dados usando StandardScaler
4. Divisão estratificada dos dados (80% treino, 20% teste)
5. Construção de uma Rede Neural com 3 camadas ocultas
6. Treinamento do modelo com early stopping (30 iterações)
7. Avaliação completa do modelo com múltiplas métricas
8. Geração de gráficos e visualizações

Dificuldades e Ajustes:

- Valores faltantes: Inicialmente causaram erro. Resolvido com imputação pela média.
- Desbalanceamento de classes: Dataset tinha 66,7% notas genuínas. Usamos stratified split.
- Implementação: Começamos com TensorFlow/Keras, mas migramos para scikit-learn por compatibilidade.

Experiência:

O projeto proporcionou excelente aprendizado sobre todo o pipeline de machine learning, desde preparação dos dados até avaliação do modelo. O uso de KNIME permitiu uma visualização clara do fluxo de processamento. A Rede Neural convergiu rapidamente (30 épocas) e mostrou excelente generalização sem sinais aparentes de overfitting. O modelo está pronto para produção.

Melhorias Futuras:

- Testar outras arquiteturas de rede (mais camadas, neurônios diferentes)
- Aplicar técnicas de aumento de dados (data augmentation)
- Implementar validação cruzada para maior robustez
- Fazer fine-tuning dos hiperparâmetros
- Analisar feature importance para entender quais variáveis são mais relevantes